

## CoMesh-sim — Gap 4 Verification Commands

Energy-Aware Coterie Membership · step-by-step commands to build and verify the change

## 0. Setup (JDK + JavaFX, offline-friendly)

```
unzip CoMesh-sim-gap4.zip
cd CoMesh-sim
```

```
sudo apt-get install -y openjdk-21-jdk-headless openjfx
JAVAFX_CP="/usr/share/openjfx/lib/javafx.base.jar:/usr/share/openjfx/lib/javafx.controls.jar:/usr/share/
```

## 1. Compile (confirms nothing is broken, no Maven Central needed)

```
mkdir -p target/classes
find src/main/java -name "*.java" > /tmp/srcs.txt
javac --release 21 -cp "$JAVAFX_CP" -sourcepath src/main/java -d target/classes @/tmp/srcs.txt
```

Expect only “unchecked/deprecated” notes, no errors.

## 2. Baseline run — confirm default behavior is unchanged

```
java -cp "target/classes:$JAVAFX_CP" -Xss4m -Xmx4096m Simulator \
-e 500 -rn 0 -dn 25 -dts grid -dtd 5,5 \
-np 0.9 -cp 0.0 -nsd uniform \
-f 1 -el 100 -rm 5 -rd 25
```

No “Gap 4” summary appears — the change is fully inert unless you opt in.

### 3. Generate an energy workload (deliberately unlucky: 70% low-energy nodes)

```
python3 workloads/scripts/createNodeEnergy.py \
-n workloads/node_list_d25_np0.9.txt -lep 0.7 -s 3 \
-o workloads/node_energy_d25_np0.9_demo.txt
```

#### 4. Run the same scenario with the Gap 4 fix enabled

```
java -cp "target/classes:$JAVAFX_CP" -Xss4m -Xmx4096m Simulator \
-e 500 -rn 0 -dn 25 -dts grid -dtd 5,5 \
-np 0.9 -cp 0.0 -nsd uniform \
-f 1 -el 100 -rm 5 -rd 25 \
-nef workloads/node_energy_d25_np0.9_demo.txt -eam
```

Look for, at the end of output:

```

---- Gap 4: Energy-Aware Coterie Membership summary ----
Coterie (re)formations checked: 138
Formations where locality-only hashing picked an all-low-energy bucket
and the energy constraint swapped a member in: 23
Formations where every online candidate was low-energy (constraint infeasible): 0

```

A non-zero “swapped a member in” count with 0 infeasible is the direct evidence the fix is doing its job.

## 5. Existing unit tests still behave the same

```

find src/test/java -name "*.java" > /tmp/test_srcs.txt
sudo apt-get install -y junit5
JUNIT_CP="/usr/share/java/junit-platform-console-standalone.jar"
mkdir -p target/test-classes
javac --release 21 -cp "target/classes:$JAVAFX_CP:$JUNIT_CP" \
    -sourcepath src/test/java -d target/test-classes @/tmp/test_srcs.txt

java -cp "target/classes:target/test-classes:$JAVAFX_CP:$JUNIT_CP" \
    org.junit.platform.console.ConsoleLauncher -c kgroup.KGroupTest --details=tree
java -cp "target/classes:target/test-classes:$JAVAFX_CP:$JUNIT_CP" \
    org.junit.platform.console.ConsoleLauncher -c dijkstra.DijkstraTest --details=tree

```

KGroupManagerTest, SimulatorTest, and network.NetworkTest fail on the original, unmodified zip too (missing workload subfolders) — that is not something this change introduced.

## 6. Isolated, code-level proof (no full simulation needed)

Drop this next to the repo and run it — it directly shows a hash-only pick landing on an all-low-energy bucket, and the energy-aware version fixing it:

```

cat > /tmp/EnergyCheck.java << 'EOF'
import kgroup.*;
import java.util.*;

public class EnergyCheck {
    public static void main(String[] args) {
        Map<String, Membership> membership = new HashMap<>();
        List<String> nodeIds = List.of("A", "B", "C", "D", "E", "F", "G");
        for (String n : nodeIds) membership.put(n, Membership.ONLINE);

        Map<String, NodeEnergyProfile> energy = new HashMap<>();
        for (String n : nodeIds) energy.put(n, new NodeEnergyProfile(10, 90)); // low energy
        energy.put("E", new NodeEnergyProfile(80, 10)); // stable
        energy.put("G", new NodeEnergyProfile(70, 20)); // stable

        int desired = 5;
        for (int trial = 0; trial < 50; trial++) {
            List<String> entities = List.of("dev" + trial);
            KGroup baseline = new DeviceKGroup(entities, membership, 1, desired, 0,
                LeaderElectionPolicy.SMALLEST_HASH, energy, false);
            List<String> pick = baseline.newKGroupMembers(membership, desired);
            if (!pick.contains("E") && !pick.contains("G")) {
                KGroup fixed = new DeviceKGroup(entities, membership, 1, desired, 0,
                    LeaderElectionPolicy.SMALLEST_HASH, energy, true);
                List<String> fixedPick = fixed.newKGroupMembers(membership, desired);
                System.out.println("baseline (all low energy): " + pick);
                System.out.println("energy-aware fix: " + fixedPick);
            }
        }
    }
}
EOF
javac --release 21 -cp "target/classes:$JAVAFX_CP" -d /tmp /tmp/EnergyCheck.java
java -cp "target/classes:/tmp:$JAVAFX_CP" EnergyCheck

```